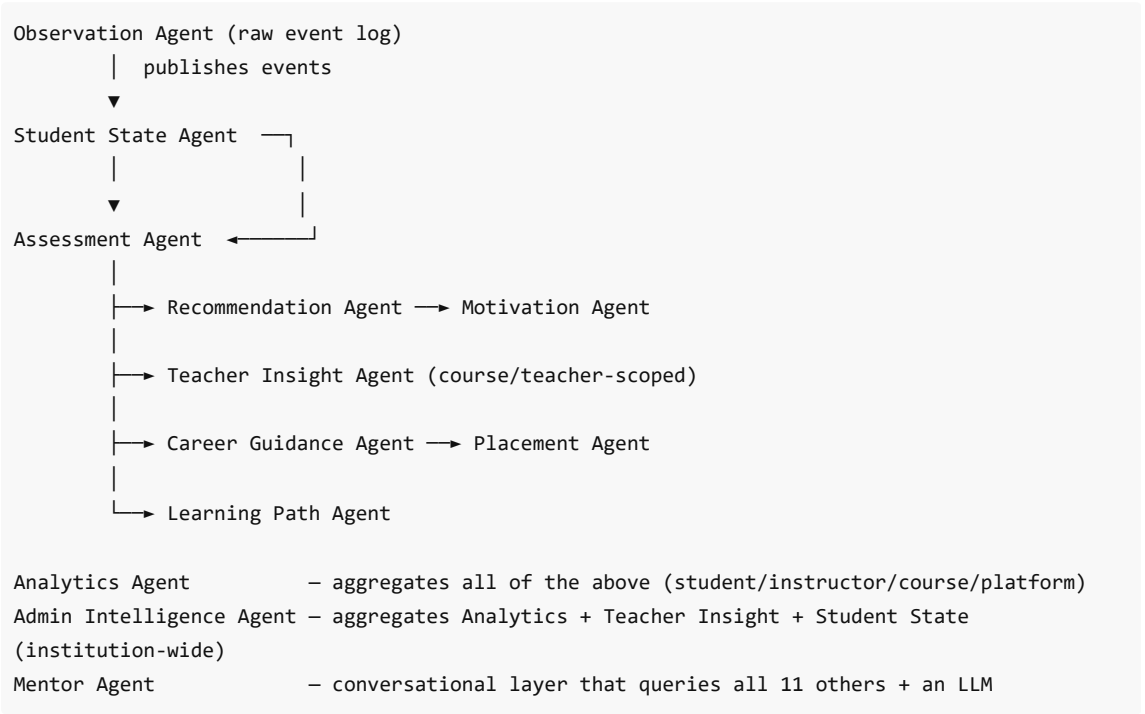
AI Agents in Orange Tree LMS

This backend contains 12 independent "AI agents" living under `lms-api/src/modules/`, plus one cross-cutting feature (the AI Student Entry Phase) that ties several of them together. Each agent is a self-contained module — its own routes, services, repositories, database tables, and event bus — that watches what happens in the LMS and turns raw activity into something useful: risk alerts, recommendations, career guidance, a chat assistant, and so on.

None of this is a separate microservice. "Agent" here means *a bounded module with a single responsibility, its own data, and a public surface other modules call into* — not a separately-deployed AI service. Only one of the twelve (the Mentor Agent) actually calls an LLM; the rest are deterministic — rule engines, scoring formulas, and statistics over real LMS data, explainable rather than black-box.

How they fit together



Every agent past Observation communicates with the others only through each module's own `index.js` (a handful of exported getters like `getFullState(studentId)`, plus `subscribe(eventName, handler)`), never by reaching into another module's internals. Each agent also owns an in-process event bus (`EventEmitter`) rather than sharing one, which keeps every module independently deployable later if needed.

Stack notes: plain JavaScript + JSDoc typedefs throughout (no TypeScript, matching the rest of `lms-api`), Prisma/PostgreSQL for storage, `node:test` for unit tests. Data model gaps against each agent's original spec (e.g. no real "Resume" or "Job Market" system) are documented per-agent below rather than faked — see "Honest gaps" in each section.

---

1. Observation Agent

`src/modules/observation` · mounted at `/events`

The event log every other agent is built on. Any part of the backend calls `publishEvent({ studentId, eventType, courseId, ... })` to record something that happened (quiz submitted, lesson completed, AI hint requested, login, etc.) — no HTTP round-trip needed, just a function call. Other agents subscribe to categories of these events to react in near-real-time.

Endpoints: `POST /events` (record one), plus several `GET` endpoints for querying the raw event history.

---

2. Student State Agent

`src/modules/student-state` · mounted at `/student-state`

Turns the raw event stream into a running picture of *where each student stands*: progress, performance, engagement, behavior (e.g. preferred learning pace), and a risk score (is this student falling behind / likely to drop off). This is the aggregate every downstream agent reads first.

Endpoints: `/dashboard`, `/progress`, `/performance`, `/engagement`, `/behavior`, `/risk`, `/course/:courseId`, `POST /recalculate`, `GET /:studentId`.

Notable: also tracks a separate per-course baseline (`StudentCourseState`) used by the AI Student Entry Phase (see below), since the main state model is global per student, not per course.

---

3. Assessment Agent

`src/modules/assessment` · mounted at `/assessment`

Owns concept-level mastery: after a quiz or assignment, it works out which specific concepts the student is weak on ("knowledge gaps"), tracks mastery trends over time, and proposes a reassessment schedule.

Endpoints: `/mastery`, `/history`, `/knowledge-gaps`, `/recommendations`, `/reassessment-plan`, `POST /evaluate`, `POST /recalculate`, `GET /:studentId`.

Also owns the AI Student Entry Phase's assessment flow at `/assessment/entry/*` — see the dedicated section below.

---

4. Recommendation Agent

`src/modules/recommendation` · mounted at `/recommendations`

Turns Assessment's knowledge gaps and Student State's risk signals into a ranked, capped list of "what to do next" items (revision, new lessons, deadline nudges), each with urgency/impact/confidence scoring so the strongest candidates win a limited number of slots rather than everything being shown at once.

Endpoints: `/today`, `/high-priority`, `/revision`, `/learning`, `POST /recalculate`, `POST /feedback`, `GET /:studentId`.

---

5. Motivation Agent

`src/modules/motivation` · mounted at `/motivation`

Watches for disengagement (streaks broken, deadlines approaching, activity dropping) and generates reminders/nudges. First agent in the series to actually consume a live peer's events in real time (Recommendation's `recommendation:updated`).

Endpoints: `/reminders`, `/streak`, `/actions`, `/history`, `POST /recalculate`, `POST /acknowledge`, `GET /:studentId`.

---

6. Teacher Insight Agent

`src/modules/teacher-insights` · mounted at `/teacher-insights`

The one agent that isn't student-scoped — it's course/instructor-scoped, aggregating every enrolled student's data for a course into one dashboard: at-risk students, course health, class performance, and weekly/monthly summaries for the instructor. Access is inverted from every other agent (only the owning instructor or an admin can see it — zero student access).

Endpoints: `/students-at-risk`, `/course-health`, `/class-performance`, `/recommendations`, `/weekly-summary`, `/monthly-summary`, `POST /recalculate`, `/course/:courseId`, `/:teacherId`.

---

7. Analytics Agent

`src/modules/analytics` · mounted at `/analytics`

Cross-cutting and explicitly **read-only** — it never modifies data or generates recommendations, only computes metrics. Covers all four scope levels (student, instructor, course, platform) through one generic schema rather than four parallel model families, and includes trend detection and simple linear-regression forecasting.

Endpoints: `/dashboard`, `/student/:studentId`, `/instructor/:instructorId`, `/course/:courseId`, `/platform`, `/kpis`, `/reports`, `/trends`, `/forecast`, `POST /recalculate`, `POST /report/export`.

Honest gap: report export is JSON/CSV only — no PDF, since this repo has no PDF-writing library.

---

8. Career Guidance Agent

`src/modules/career` · mounted at `/career`

Maps a student's demonstrated skills (from Assessment's concept mastery) against industry roles, surfaces skill gaps, and builds a roadmap toward a chosen career goal.

Endpoints: `/profile/:studentId`, `/readiness`, `/roles`, `/roadmap`, `/skill-gaps`, `/recommendations`, `/interview-plan`, `POST /recalculate`, `POST /goal`.

Honest gaps: there's no real "Skills"/"Projects"/"external Certifications" system in this LMS — technical skills are derived from Assessment's mastery data, and the "Industry Skill Taxonomy" is this agent's own seeded reference dataset (10 roles). "Job Market Trends" is a real adapter (`ai/jobMarketProvider.js`) currently backed by the same static seed data, ready to swap for a real provider (LinkedIn/Indeed/etc.) with no other code changes.

---

9. Learning Path Agent

`src/modules/learning-path` · mounted at `/learning-path`

Personalizes the order and pacing of lessons: what to study next, daily/weekly study plans, and pace adjustments based on how fast or slow a student is actually moving.

Endpoints: `/next`, `/daily-plan`, `/weekly-plan`, `/recommendations`, `/milestones`, `POST /recalculate`, `GET /:studentId`.

Honest gap: there's no prerequisite-graph model in this LMS — module/lesson ordering already functions as the de facto prerequisite chain, so "prerequisite" means "the next incomplete lesson in sequence," not a dependency DAG.

---

10. Placement Agent

`src/modules/placement` · mounted at `/placement`

Career Guidance's structural sibling, aimed at job/internship readiness: matches students against a job/internship catalog, tracks applications and interview outcomes, and scores interview/placement readiness.

Endpoints: `/jobs`, `/internships`, `/drives`, `/matches`, `/applications`, `/interviews`, `/offers`, `POST /recalculate`, `POST /application`, `/profile/:studentId`.

Honest gaps: `Company` / `JobOpportunity` / `InternshipOpportunity` / `PlacementDrive` are this agent's own seeded catalog (6 companies, 9 jobs, 5 internships, 2 drives) rather than a real job board. The external listings adapter (`integrations/jobPortalProvider.js`) honestly returns an empty result set rather than faking a live feed. `resumeQualityScore` / `portfolioQualityScore` are proxies (credential count, skill breadth, profile completeness) since there's no actual resume/portfolio content anywhere to analyze.

---

11. Admin Intelligence Agent

`src/modules/admin-intelligence` · mounted at `/admin-intelligence`

The institution-wide view, sitting one layer above Analytics: department/faculty performance, compliance auditing of the other agents' own AI outputs, capacity forecasting (enrollment, course capacity, instructor load), and strategic recommendations/alerts for admins. ADMIN-only.

Endpoints: `/dashboard`, `/institution-health`, `/departments`, `/faculty`, `/student-risk`, `/compliance`, `/reports`, `/forecasts`, `/alerts`, `POST /recalculate`, `POST /report/export`.

Honest gaps: there's no real `Department` / `Faculty` / `Semester` model in this schema — "department" is `Course.category`, "faculty" is just instructor users. `ComplianceAudit` is a genuine append-only ledger that scans Teacher Insight's real confidence scores for anomalies (this build actually found and fixed a real pre-existing bug: a missing `confidenceScore` field on `CourseInsight`).

---

12. Mentor Agent (AI Chat Assistant)

`src/modules/mentor` · mounted at `/mentor`

The conversational layer on top of everything else, and the only agent that calls an LLM. A student, instructor, or admin can chat with it; it detects intent, queries whichever peer agents are relevant to that intent and role, merges their answers with source attribution, and replies — via Anthropic's API when `ANTHROPIC_API_KEY` is configured, or an honest non-generative fallback (built from the same real data, just not LLM-phrased) when it isn't.

Endpoints: `POST /chat`, `POST /stream` (Server-Sent Events), `/history`, `/context`, `/recommendations`, `/conversation/:id` (get/delete), `POST /feedback`.

Notable: no single agent call is allowed to hang or crash a chat turn — every peer-agent call is wrapped with an 8s timeout and always resolves to a well-formed result. Memory is two-tier: recent messages ride verbatim, older ones compact into a summary, and a small fixed set of facts (last intent, career goal, etc.) persists across conversations.

AI Student Entry Phase

(not a 13th agent — a cross-cutting flow spanning `StudentProfile`, Assessment, Student State, and Learning Path)

Triggered by course enrollment rather than one of the 12 agents' own event buses:

1. Student enrolls in a course.
2. A 15-question entry assessment (5 easy / 5 medium / 5 hard) is generated, scoped to that course's actual module titles — LLM-generated when `ANTHROPIC_API_KEY` is set, otherwise sampled from real questions already attached to the course's quizzes. If neither can produce enough real questions, the assessment is honestly marked `FAILED` rather than inventing MCQs with made-up correct answers.
3. Answers are evaluated into per-concept mastery (`GET/POST /assessment/entry/:courseId/...`).
4. Student State initializes a per-course baseline (`StudentCourseState`) and assigns a per-module learning mode — Smart Revision / Standard / Deep Learning — based on mastery thresholds.
5. Learning Path personalizes each lesson's duration from that mode (compressing familiar material down to a 5-minute floor, expanding weak areas up to 1.5×).

There's an explicit "skip for now" escape hatch on the frontend so a failed or unavailable assessment never blocks course access.

Shared architectural pattern (every agent, no exceptions)

- Own `EventEmitter` -based pub/sub bus per module (not shared) — keeps every agent independently deployable.
- Own copies of near-identical middleware (`resolveStudentAccess` , `validateQuery`) rather than a shared one, for the same reason.
- Layering: `constants/ types/ utils/ services/ repositories/ dto/ events/ schedulers/ middleware/ validators/ controllers/ routes/ tests/` . Pure domain logic lives in `services/domain/` , separated from I/O, so it's unit-testable without a database.
- Cross-agent reads only through another module's `index.js` public surface (`subscribe` + a handful of named getters) — never by importing another agent's internals directly.
- Each new agent's `bootstrap()` is called once in `server.js` ; its router is mounted in `app.js` .
- Where a spec claims a system that doesn't exist in this codebase (a "Resume" model, a "Job Market" feed, a "Prerequisite" graph, etc.), the gap is filled with the most honest minimal real thing — a seeded reference dataset, a real adapter with an empty default, or an existing field repurposed — and stated plainly rather than fabricated. See the "Honest gaps" note under each agent above.